

GUÍA DE USO — DEMOSTRACIÓN INTERACTIVA

Simulador CPU-OS

Procesos · Scheduling · PCB · Deadlock

MF0490_3 — UD2 · IFCT0109 Seguridad Informática

Módulo	MF0490_3 — Gestión de Servicios en el Sistema Informático
Unidad	UD2 — Análisis y Gestión de Procesos del Sistema
Simulador	CPU-OS Simulator (React) — funciona en navegador, Guacamole y móvil
Simulador externo	https://cpu-os-simulator.com (complementario)
Conceptos	Estados de proceso · Scheduling · PCB / Context Switch · Deadlock

juanprofesor.com | [Laboratorio Guacamole](#) | IFCT0109

¿Por qué usar un simulador?

Los conceptos de gestión de procesos del sistema operativo son abstractos: es difícil visualizar cómo el scheduler decide el orden de ejecución, cómo se guarda y restaura el contexto de un proceso o por qué cuatro procesos pueden quedar bloqueados indefinidamente. El simulador convierte estos conceptos en experiencias visuales e interactivas antes de trabajarlos en el sistema real.

Simulador propio (React)

El simulador integrado en el cuaderno digital funciona directamente en el navegador, sin instalación. Incluye 4 módulos: (1) Estados de proceso con máquina de estados animada, (2) Algoritmos de scheduling con diagrama de Gantt, (3) PCB y context switch paso a paso, (4) Deadlock con grafo de asignación de recursos y condiciones de Coffman.

Simulador externo complementario

cpu-os-simulator.com ofrece simulaciones adicionales de memoria virtual, paginación y segmentación que complementan el simulador propio. Se recomienda usarlo después de los módulos 1-4 del simulador React para profundizar en gestión de memoria.

Módulo 1 — Estados de Proceso

Conceptos clave

Un proceso pasa por 5 estados durante su ciclo de vida: NEW (creado, aún no en memoria), READY (listo para ejecutar, esperando CPU), RUNNING (ejecutándose en la CPU), WAITING (bloqueado esperando E/S u otro evento) y TERMINATED (ha finalizado su ejecución). Las transiciones entre estados las gestiona el SO.

Diagrama de transiciones

Transición	Evento que la provoca	Descripción
NEW → READY	Admisión por el SO	El proceso es cargado en memoria y puesto en la cola de listos.
READY → RUNNING	Dispatch del scheduler	El scheduler selecciona el proceso y le asigna la CPU.
RUNNING → WAITING	Solicitud de E/S o evento	El proceso solicita una operación de E/S y se bloquea hasta que se complete.
RUNNING → READY	Preempción / quantum agotado	El scheduler preempta el proceso (expira su quantum en RR o llega proceso de mayor prioridad).
WAITING → READY	E/S completada / interrupción	El evento esperado se ha completado. El proceso vuelve a la cola de listos.
RUNNING → TERMINATED	Llamada exit() o señal	El proceso ha terminado su ejecución o ha sido terminado por el SO/usuario.

Uso del simulador — Estados

Paso a paso en el simulador

1. Abre la pestaña 'Estados' del simulador. Observa el diagrama de máquina de estados a la izquierda y la tabla de procesos a la derecha.
2. Pulsa '► Auto-simular' para ver cómo los procesos transicionan automáticamente.
3. Pausa la simulación y usa los botones de transición manual (→new, →rea, →run...) de cada proceso.
4. Añade nuevos procesos con '+ Proceso'.
5. Observa cómo el diagrama de estados se ilumina según el número de procesos en cada estado.

Pregunta 1: ¿Cuántos procesos pueden estar simultáneamente en estado RUNNING en un sistema con un solo núcleo de CPU? ¿Y en un sistema con 4 núcleos? Razona la respuesta.

Pregunta 2: Observa los procesos en estado WAITING en el simulador. ¿Qué evento necesita producirse para que un proceso pase de WAITING a READY? ¿Quién genera ese evento en un sistema real?

Pregunta 3: ¿Qué diferencia hay entre un proceso TERMINATED y uno que sigue apareciendo en la lista pero ha terminado (proceso zombie)? ¿Qué comando de Linux usarías para detectarlo?

Conexión con el sistema real — DC01 y ubuntu-moderna

Verifica en el laboratorio

En ubuntu-moderna: ejecuta 'ps aux' y observa la columna STAT. Identifica procesos en estado S (sleeping/waiting), R (running), D (disk sleep) y Z (zombie). En DC01: abre Task Manager → pestaña Detalles → columna Estado. Compara los estados con los del simulador.

Módulo 2 — Algoritmos de Scheduling

Conceptos clave

El scheduler (planificador de CPU) decide qué proceso de la cola de listos se ejecuta a continuación. Los algoritmos difieren en criterios de selección, equidad y rendimiento. Las métricas clave son: Tiempo de espera (WT = inicio_ejecución - llegada), Tiempo de retorno o TAT (turnaround = fin - llegada) y Uso de CPU (%).

Algoritmo	Preemptivo	Inanición	Característica principal
FIFO / FCFS	No	No	Orden de llegada. Simple. Efecto convoy si hay procesos largos.
SJF	No (SRTF sí)	Posible	Mínimo WT medio teórico. Requiere conocer el burst time.
Round Robin	Sí (quantum)	No	Equitativo. Quantum pequeño → más context switches. Quantum grande → FIFO.
Prioridad	Sí o No	Sí (aging)	Proceso de mayor prioridad primero. Aging evita inanición de baja prioridad.

Uso del simulador — Scheduling

Paso a paso en el simulador

1. Abre la pestaña 'Scheduling'. Observa los 5 procesos de la cola con su burst time y prioridad.
2. Selecciona el algoritmo FIFO y pulsa '► Simular'. Observa el diagrama de Gantt generado. Anota el tiempo total y las métricas.
3. Sin cambiar los procesos, cambia a SJF y simula de nuevo. Compara el WT medio con FIFO.
4. Prueba Round Robin con quantum=2, luego con quantum=5. Observa cómo cambia el número de bloques en el Gantt.
5. Prueba Prioridad y observa qué proceso se ejecuta primero.
6. Pulsa '↺ Reiniciar' para generar una nueva cola aleatoria y repetir.

Tabla comparativa de algoritmos (rellena con los valores del simulador)

Algoritmo	Tiempo total	WT medio	TAT medio	Uso CPU %
FIFO	—	—	—	—
SJF	—	—	—	—
Round Robin (Q=2)	—	—	—	—
Round Robin (Q=5)	—	—	—	—
Prioridad	—	—	—	—

Pregunta 4: Según los datos de tu tabla: ¿qué algoritmo tiene el menor tiempo de espera medio? ¿Coincide con la teoría (SJF es óptimo en WT)? Si hay diferencias, ¿a qué se deben?

Pregunta 5: ¿Qué problema práctico tiene SJF que lo hace difícil de implementar en un SO real? ¿Cómo lo resuelven los schedulers modernos (CFS de Linux)?

Pregunta 6: *Compara Round Robin con quantum=2 y quantum=5. ¿Cuál genera más bloques en el diagrama de Gantt? ¿Qué implicación tiene esto en el rendimiento del sistema?*

Relación con el sistema real

Verifica en el laboratorio

En ubuntu-moderna: 'cat /proc/schedstat' muestra estadísticas del scheduler CFS. 'chrt -p <PID>' muestra la política de scheduling de un proceso. En DC01: 'Get-Process | Select-Object Name, PriorityClass' muestra la clase de prioridad de cada proceso. Compara con lo visto en el simulador.

Módulo 3 — PCB y Cambio de Contexto

Conceptos clave

El PCB (Process Control Block) es la estructura de datos del kernel que almacena TODA la información sobre un proceso: PID, estado, registros de CPU, contador de programa, puntero de pila, tabla de ficheros abiertos, información de memoria y estadísticas. El context switch (cambio de contexto) es la operación de guardar el PCB del proceso actual y cargar el PCB del siguiente proceso en la CPU.

Campos del PCB

Campo	Tipo de dato	Descripción
PID	Entero (32/64 bits)	Identificador único del proceso en el sistema.
Estado	Enumeración	New / Ready / Running / Waiting / Terminated.
Program Counter (PC)	Puntero (dirección)	Dirección de la siguiente instrucción a ejecutar.
Registros CPU	Array de registros	AX, BX, CX, DX, SP, BP, FLAGS, registros de coma flotante...
Información de memoria	Tabla de páginas / segmentos	Mapeo de memoria virtual a física del proceso.
Tabla de ficheros abiertos	Array de descriptores	Lista de ficheros y sockets abiertos por el proceso.
Información de E/S	Lista de dispositivos	Dispositivos de E/S asignados y solicitudes pendientes.
Información de planificación	Prioridad, quantum...	Datos que usa el scheduler para decidir la ejecución.
PID padre (PPID)	Entero	PID del proceso que creó este proceso (fork).

Uso del simulador — PCB y Context Switch

Paso a paso en el simulador

1. Abre la pestaña 'PCB / Ctx'. Observa los dos PCBs mostrados: PID 101 (nginx, RUNNING) y PID 102 (mysqld, READY). 2. Examina todos los campos de cada PCB: registros de CPU (PC, SP, AX, BX, CX, FLAGS), estado, prioridad, burst, ficheros abiertos y PID padre. 3. Pulsa 'Context Switch'. Observa cómo la animación muestra los 3 pasos: guardar contexto del proceso actual → actualizar estado → cargar contexto del siguiente proceso. 4. Observa que al finalizar, el panel 'EN CPU' cambia de nginx a mysqld. 5. Compara los valores de PC y SP entre los dos procesos — son completamente diferentes porque cada proceso tiene su propio espacio de dirección.

Pregunta 7: ¿Por qué es necesario guardar el Program Counter (PC) en el PCB durante un context switch? ¿Qué ocurriría si no se guardara?

Pregunta 8: El context switch tiene un coste en tiempo (típicamente 1-10 μ s en hardware moderno). ¿Cómo influye este coste en la elección del quantum en Round Robin? ¿Qué pasaría si el quantum fuera de 1 μ s?

Pregunta 9: *Observa que nginx tiene 3 ficheros abiertos y mysqld tiene 12. ¿Qué información del PCB necesita el SO para restaurar estos descriptores de fichero al hacer el context switch?*

Relación con el sistema real

Verifica en el laboratorio

En ubuntu-moderna: 'cat /proc/<PID>/status' muestra el PCB de un proceso real. 'cat /proc/<PID>/fdinfo/' lista los descriptores de fichero abiertos. 'ls -la /proc/<PID>/fd' muestra qué ficheros tiene abiertos. En DC01: 'Get-Process -Id <PID> | Format-List *' muestra todos los campos del equivalente Windows del PCB.

Módulo 4 — Interbloqueo (Deadlock)

Conceptos clave

El deadlock o interbloqueo es una situación en la que un conjunto de procesos queda bloqueado indefinidamente porque cada proceso retiene recursos que otros procesos del conjunto necesitan. Para que ocurra un deadlock es necesario y suficiente que se cumplan **SIMULTÁNEAMENTE** las 4 condiciones de Coffman (1971).

Las 4 condiciones de Coffman

#	Condición	Descripción	Cómo romperla (prevención)
1	Exclusión mutua	El recurso solo puede ser usado por un proceso a la vez.	Usar recursos compartibles (ej: ficheros de solo lectura). No siempre posible.
2	Retención y espera	Un proceso retiene recursos y espera otros adicionales.	Exigir que los procesos soliciten todos los recursos al inicio (all-or-nothing).
3	No preempción	Los recursos no pueden ser quitados forzosamente a un proceso.	Permitir preempción: si un proceso no puede obtener un recurso, libera los que tiene.
4	Espera circular	Existe un ciclo $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$ en el grafo de asignación de recursos.	Imponer un orden global de solicitud de recursos (numeración). Elimina ciclos.

Uso del simulador — Deadlock

Paso a paso en el simulador

1. Abre la pestaña 'Deadlock'. Selecciona el escenario 'Sin deadlock'. Observa el grafo de asignación de recursos: las líneas continuas indican 'recurso asignado a proceso' y las discontinuas 'proceso esperando recurso'. Verifica que no hay ciclos en el grafo. 2. Selecciona 'Deadlock circular'. Observa cómo el grafo forma un ciclo $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$. Verifica que las 4 condiciones de Coffman aparecen como cumplidas. 3. Selecciona 'Espera parcial'. Solo P1 y P2 están en deadlock; P3 y P4 pueden continuar. 4. En los 3 escenarios, revisa el panel 'Estrategias de resolución'.

Pregunta 10: Observa el escenario 'Deadlock circular' del simulador. Dibuja o describe el ciclo de espera entre los 4 procesos. ¿Qué recurso necesita liberar para romper el deadlock con el mínimo impacto?

Pregunta 11: ¿Por qué el escenario 'Espera parcial' NO es un deadlock completo? ¿Qué diferencia hay entre un deadlock y un livelock?

Pregunta 12: El Algoritmo del Banquero (Dijkstra, 1965) evita el deadlock sin romper ninguna condición de Coffman. ¿Cuál es su limitación práctica principal para implementarlo en un SO moderno?

Deadlock en sistemas reales

Verifica en el laboratorio

En ubuntu-moderna: 'cat /proc/locks' muestra los bloqueos de ficheros activos. Wazuh puede detectar procesos en estado D (disk sleep) prolongado que pueden indicar deadlock de E/S. En DC01: el Event Viewer → Application registra timeouts de transacciones que pueden ser síntoma de deadlock en bases de datos.

Deadlock en producción

Los deadlocks en sistemas de bases de datos (MySQL, SQL Server) son frecuentes y el motor los detecta y resuelve automáticamente terminando la transacción 'víctima'. En sistemas embebidos y tiempo real, el deadlock puede ser fatal. Windows implementa el 'Wait Chain Traversal' (WCT) para detectar deadlocks de aplicaciones en el Task Manager → Pestaña Rendimiento.

Actividad Integradora — Informe de Demostración

Tras completar los 4 módulos del simulador, elabora un informe breve que integre los conceptos trabajados. El informe debe demostrar que comprendes cómo los conceptos teóricos se aplican en un sistema operativo real.

Estructura del informe (máximo 2 páginas A4)

#	Sección	Contenido requerido
1	Estados de proceso	Diagrama de transiciones elaborado a mano o digital. Descripción de un proceso real del laboratorio y los estados por los que pasa desde su inicio hasta su terminación.
2	Comparativa de algoritmos	Tabla comparativa completada con los datos del simulador. Conclusión sobre qué algoritmo elegiría para un servidor web (nginx) y por qué.
3	Análisis de un PCB real	Captura de 'cat /proc/<PID>/status' de un proceso en ubuntu-moderna. Identificación de los campos equivalentes a los del simulador.
4	Escenario de deadlock	Descripción de un escenario real en el que podría producirse un deadlock en el laboratorio IFCT0109. ¿Entre qué procesos? ¿Qué recursos? ¿Cómo se prevendría?

ENTREGABLE

- Tabla comparativa de algoritmos (Módulo 2) con los valores reales del simulador completados.
- Respuestas razonadas a las 12 preguntas de los 4 módulos.
- Informe integrador (máximo 2 páginas) con las 4 secciones descritas.
- Captura de 'cat /proc/<PID>/status' de un proceso real en ubuntu-moderna.
- OPCIONAL: Captura del simulador externo cpu-os-simulator.com mostrando un ejercicio de scheduling adicional.

